UNIVERSIDAD POLITÉCNICA
BICENTENARIO

Práctica 3: Control ON-OFF de temperatura.

Presenta:

Rubén Antonio Arredondo Villegas

Edgar ivan Gutierrez Gutierrez

Roberto Leonel Mendoza Nuñez

Ailton Alejandro Trujillo López

Matricula:

24033101

24032957

24033370

24033482

Asignatura:

Sistemas embebidos

Profesor:

Hugo Antonio Méndez Guzmán

Silao, Gto. A13 de octubre del 2026

Contenido

Introducción 3

Marco Teórico 3

Desarrollo 3

Resultados 13

Observaciones y Conclusiones 17

Referencias 17

Lista de Figuras

Ilustración 1. Circuito de encendido y apago del foco y celda peltier 6

Ilustración 2Circuito todo ensamblado y en ejecución 14

Ilustración 3Circuito en ejecución enfriando y calentado visto todo ensamblado 15

Ilustración 4Pantalla lcd con los máximos y mínimos de enfriamiento y calentamiento ... 16

Introducción

Los sistemas de automatización industrial requieren que un controlador pueda tomar decisiones a partir de diferentes condiciones de entrada. Una de las formas más sencillas de representar estas condiciones es mediante las operaciones lógicas AND y OR, ampliamente utilizadas en circuitos digitales, sistemas de control, PLC y automatización electroneumática.

En esta práctica se realizó un sistema de control de temperatura utilizando una ESP32, un sensor de temperatura, una celda Peltier, ventiladores y un foco. El objetivo fue mantener la temperatura de una caja entre 30 °C y 40 °C, haciendo que el sistema activara automáticamente el enfriamiento o el calentamiento según fuera necesario. De esta manera, se pudo observar cómo funciona un proceso automatizado de control de temperatura utilizando componentes electrónicos y programación.

Marco Teórico

El ESP32 es un microcontrolador que integra entradas y salidas digitales, comunicación inalámbrica y múltiples periféricos que permiten desarrollar sistemas de control y automatización [1]. Para estas prácticas se utilizan principalmente sus pines GPIO (*General Purpose Input/Output*), configurados como entradas digitales para detectar el estado lógico de los botones de encendido y apagado, y como salidas digitales para enviar señales de control [1].

Señales e Entradas/Salidas Digitales

Una señal digital se representa mediante dos niveles de voltaje correspondientes a estados lógicos discretos [2]:

- **LOW (0):** Nivel lógico bajo (0 V).
- **HIGH (1):** Nivel lógico alto (3.3 V en la arquitectura del ESP32).

En el ESP32, una entrada digital determina si un pulsador (*push button*) está presionado o liberado mediante la lectura de voltaje en el pin GPIO asignado [1], [2].

Además del ESP32, el sistema está compuesto por diferentes dispositivos que permiten medir la temperatura, mostrar información al usuario y controlar el proceso de enfriamiento.

La pantalla LCD 16x2 se utiliza para mostrar la temperatura detectada por el sensor y el estado de funcionamiento del sistema. Gracias a ella, el usuario puede visualizar la información en tiempo real sin necesidad de conectar el microcontrolador a una computadora.

Para la medición de temperatura se emplea el sensor DS18B20, el cual proporciona lecturas digitales precisas y mantiene una comunicación directa con el ESP32 mediante el protocolo One-Wire. La información obtenida por este sensor es utilizada para tomar decisiones sobre el encendido o apagado del sistema de enfriamiento.

El enfriamiento se realiza mediante una celda Peltier TEC1-12706, un dispositivo termoeléctrico capaz de generar una diferencia de temperatura entre sus caras cuando es alimentado con corriente eléctrica. De esta manera, una cara absorbe calor y produce enfriamiento, mientras que la otra libera calor al ambiente.

Para controlar el funcionamiento de los elementos de potencia se utiliza un módulo de relés **de** 4 canales, que permite activar o desactivar dispositivos de mayor consumo eléctrico mediante señales enviadas por el ESP32.

Asimismo, el sistema cuenta con dos ventiladores de 12 V de 40 mm × 40 mm, cuya función es disipar el calor generado por la cara caliente de la celda Peltier, evitando que ésta pierda eficiencia durante su operación. También se utiliza un ventilador de 5 V de 30 mm × 30 mm **×** 10 mm, encargado de distribuir el aire frío en la zona de enfriamiento.

Finalmente, una fuente de alimentación de 12 V proporciona la energía necesaria para el funcionamiento de la celda Peltier, los ventiladores y los demás componentes del sistema, garantizando una operación estable y segura.

Material requerido

- 1 tarjeta ESP32.
- 3 botones pulsadores.
- 1 resistencias de 4.7 kΩ.
- 1 foco.
- 1 resistencia de 220 Ω o 330 Ω.
- Protoboard.
- Cables Dupont.
- 1 pantalla lcd (16x2)
- 1 celda peltier (TEC1-12706)
- 1 sensor DS18B20
- 1 módulo de relés de 4 canales (5v)
- 2 ventiladores (12v 4x4)
- 1 ventilador (5v **30 mm x 30 mm x 10 mm)**
- Fuente de 12 v

- Cable USB.
- Computadora con Arduino IDE.
- Asignación de pines

Desarrollo

Al comenzar la práctica, primero se revisaron todos los componentes para verificar que funcionaran de manera adecuada. Una de las primeras actividades fue comprobar la dirección del flujo de aire de los ventiladores, con el fin de colocarlos correctamente tanto en la caja como en el sistema de enfriamiento de la celda Peltier. Asimismo, durante el armado de la celda Peltier se verificó cuál de sus caras generaba frío y cuál disipaba calor, ya que una instalación incorrecta habría requerido desmontar nuevamente todo el sistema.

Una vez ensamblada la celda Peltier, se procedió a tomar las medidas necesarias para realizar los recortes en la caja y montar cada uno de los componentes utilizados. Después de obtener las medidas correspondientes, se inició el corte de la parte superior de la caja, lugar donde se instaló la celda Peltier. Esta ubicación se eligió para permitir que el aire frío se distribuyera de manera uniforme dentro del compartimiento.

Posteriormente, se realizó el corte lateral para la instalación de uno de los ventiladores. Su función principal era favorecer la circulación del aire dentro de la caja y ayudar a expulsar el aire caliente generado durante el funcionamiento del sistema. En la parte superior también se colocó el sensor de temperatura DS18B20, encargado de monitorear constantemente la temperatura interna. Gracias a este sensor, el ESP32 podía determinar cuándo apagar el foco calefactor y activar la celda Peltier para mantener la temperatura dentro del rango establecido.

El foco se instaló mediante un soquet para garantizar una conexión segura y facilitar su reemplazo en caso necesario. Este elemento fue colocado en el interior de la caja con el objetivo de generar calor. Para su control, los cables del foco se conectaron al módulo de relés, mientras que la alimentación eléctrica provenía directamente de la corriente de entrada.

Por otra parte, la pantalla LCD fue montada sobre un protoboard junto con el resto de las conexiones electrónicas. Esta pantalla permitió visualizar en tiempo real la temperatura registrada por el sensor, así como el comportamiento del sistema durante las pruebas, mostrando cuándo la temperatura aumentaba o disminuía.

Finalmente, la fuente de alimentación de 12 V fue utilizada para suministrar energía a la celda Peltier y a los ventiladores del sistema. La pantalla LCD y el ESP32 fueron

alimentados mediante la conexión USB de la computadora, lo que facilitó tanto la programación como el monitoreo de los datos durante el desarrollo de la práctica.

Tuvimos un pequeño detalle ya que nuestra celda peltier no venía con el ventilador que dispersaba el aire frio dentro dela caja y el cual compramos al día siguiente y lo instalamos con una pequeña adaptación ya que en la parte de la celda tampoco venia un espacio para fijarlo bien.

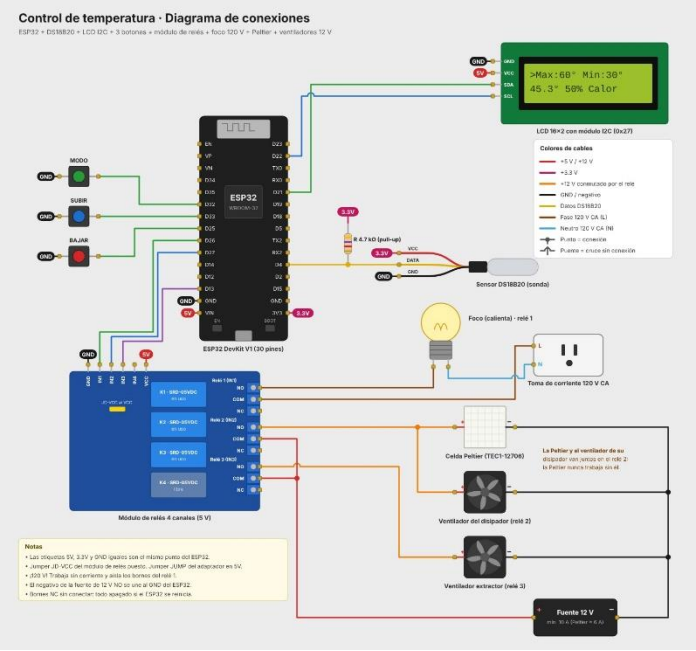


Ilustración 1. Circuito de encendido y apago del foco y celda peltier

El diagrama mostrado anteriormente representa la conexión completa de todos los componentes que integran el sistema de control de temperatura. En él se puede observar la interacción entre el ESP32, el sensor de temperatura DS18B20, la pantalla LCD, el módulo de relés, la celda Peltier, los ventiladores y la fuente de alimentación.

El ESP32 funciona como la unidad principal de control, ya que recibe la información de temperatura proporcionada por el sensor DS18B20 y procesa los datos para tomar decisiones de acuerdo con la programación establecida. Dependiendo de la temperatura detectada, el microcontrolador envía señales al módulo de relés para activar o desactivar el foco calefactor y la celda Peltier.

A continuación, se muestra el código empleado en el IDLE de arduino.

```
//
     Botones:
   - MODO  -> cada pulsacion cambia entre editar el MAX y editar el MIN.
   - SUBIR -> sube 1 grado el valor seleccionado.
   - BAJAR -> baja 1 grado el valor seleccionado.
   Los valores se guardan en memoria y se conservan aunque se vaya la luz.
//

#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Preferences.h>

// ---------------------- CONFIGURACION DE PINES ----------------------
#define SENSOR_PIN        4    // DATA del DS18B20

#define RELAY_FOCO        26   // IN1 - foco (CALIENTA)
#define RELAY_PELTIER     27   // IN2 - celda Peltier + su ventilador (ENFRIA)
#define RELAY_EXTRACTOR   13   // IN3 - ventilador extractor (solo al ENFRIAR)

#define BTN_MODO          32
#define BTN_SUBIR         33
#define BTN_BAJAR         25

#define LCD_SDA           21
#define LCD_SCL           22

// La mayoria de modulos de rele son "activos en BAJO" (LOW).
// Si tus reles funcionan al reves, cambia LOW por HIGH.
#define RELAY_ACTIVO   LOW
#define RELAY_INACTIVO (!RELAY_ACTIVO)

// ---------------------- PARAMETROS DE CONTROL ---------------------------
const int TEMP_MAX_INICIAL = 60;   // C - valor de fabrica del MAX
const int TEMP_MIN_INICIAL = 30;   // C - valor de fabrica del MIN
const int LIMITE_INFERIOR  = 0;    // C - lo mas bajo que se puede configurar
const int LIMITE_SUPERIOR  = 99;   // C - lo mas alto que se puede configurar
const int PASO_SETPOINT    = 1;    // grados que cambia el MAX/MIN por pulsacion
const int MARGEN_MINIMO    = 5;    // separacion minima entre MAX y MIN (grados)

const unsigned long PAUSA_CAMBIO_MS = 1000;  // foco y Peltier apagados al
cambiar
const int MAX_ERRORES_SENSOR = 3;            // lecturas fallidas seguidas antes
de parar

// ---------------------- OBJETOS GLOBALES --------------------------------
LiquidCrystal_I2C lcd(0x27, 16, 2);  // si no se ve nada, prueba 0x3F
OneWire oneWire(SENSOR_PIN);
DallasTemperature sensor(&oneWire);
Preferences prefs;

// ---------------------- ESTADO DEL SISTEMA ------------------------------
enum EstadoCiclo { INICIANDO, CALENTANDO, ENFRIANDO, ERROR_SENSOR };
EstadoCiclo estado = INICIANDO;
```

```
int tempMax = TEMP_MAX_INICIAL;
int tempMin = TEMP_MIN_INICIAL;

enum ModoEdicion { EDITAR_MAX, EDITAR_MIN };
ModoEdicion modoEdicion = EDITAR_MAX;

float tempActual    = 0.0;
float porcentaje    = 0.0;
int   erroresSensor = 0;
bool  hayLectura    = false;

unsigned long momentoCambioEstado = 0;

// Lectura del sensor sin bloquear el programa
unsigned long momentoPeticion = 0;
const unsigned long TIEMPO_CONVERSION_MS = 800;

// Botones (deteccion de flanco + antirrebote)
bool anteriorModo  = HIGH;
bool anteriorSubir = HIGH;
bool anteriorBajar = HIGH;
unsigned long ultimoPulsoModo  = 0;
unsigned long ultimoPulsoSubir = 0;
unsigned long ultimoPulsoBajar = 0;
const unsigned long DEBOUNCE_MS = 200;

unsigned long ultimaActualizacionLCD = 0;
const unsigned long INTERVALO_LCD_MS = 300;

const char GRADO = (char)223;  // simbolo de grados en la LCD

// ---------------------- DECLARACION DE FUNCIONES -----------------------
void leerSensor();
void actualizarCiclo();
void cambiarEstado(EstadoCiclo nuevo);
void aplicarSalidas();
void apagarFocoYPeltier();
void actualizarExtractor();
const char* nombreEstado();
bool botonPresionado(int pin, bool &anterior, unsigned long &ultimoPulso);
void manejarBotones();
void validarLimites();
void guardarConfiguracion();
void actualizarLCD();
void imprimirLinea(int fila, const char* texto);

// ============================================================================
void setup() {
  Serial.begin(115200);

  pinMode(RELAY_FOCO, OUTPUT);
  pinMode(RELAY_PELTIER, OUTPUT);
  pinMode(RELAY_EXTRACTOR, OUTPUT);
  apagarFocoYPeltier();
  digitalWrite(RELAY_EXTRACTOR, RELAY_INACTIVO);  // el extractor arranca apagado

  pinMode(BTN_MODO,  INPUT_PULLUP);
  pinMode(BTN_SUBIR, INPUT_PULLUP);
  pinMode(BTN_BAJAR, INPUT_PULLUP);

  // LCD
  delay(500);
  Wire.begin(LCD_SDA, LCD_SCL);
```

```
  Wire.setClock(100000);
  lcd.init();
  lcd.backlight();
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Iniciando...");

  // Sensor DS18B20 (modo sin espera para no congelar los botones)
  sensor.begin();
  sensor.setResolution(12);              // 0.0625 C de resolucion
  sensor.setWaitForConversion(false);
  if (sensor.getDeviceCount() == 0) {
    Serial.println("AVISO: no se detecto ningun DS18B20. Revisa DATA y la
resistencia de 4.7k.");
  }
  sensor.requestTemperatures();
  momentoPeticion = millis();

  // Cargar MAX y MIN guardados (si no hay nada guardado: 60 y 30)
  prefs.begin("configTemp", false);
  tempMax = prefs.getInt("tempMax", TEMP_MAX_INICIAL);
  tempMin = prefs.getInt("tempMin", TEMP_MIN_INICIAL);
  prefs.end();
  validarLimites();

  Serial.println("=== Control de temperatura - ciclo continuo ===");
  Serial.print("MAX: "); Serial.print(tempMax); Serial.print(" C  MIN: ");
Serial.print(tempMin); Serial.println(" C");

  delay(1000);
  lcd.clear();
}

// ============================================================================
void loop() {
  manejarBotones();

  // Cuando el sensor termina de medir: leer, actualizar el ciclo y pedir otra
medicion
  if (millis() - momentoPeticion >= TIEMPO_CONVERSION_MS) {
    leerSensor();
    actualizarCiclo();
    sensor.requestTemperatures();
    momentoPeticion = millis();
  }

  if (millis() - ultimaActualizacionLCD >= INTERVALO_LCD_MS) {
    ultimaActualizacionLCD = millis();
    actualizarLCD();
  }
}

// ============================================================================
// SENSOR DS18B20
void leerSensor() {
  float t = sensor.getTempCByIndex(0);

  // -127 = sensor desconectado. 85 en la primera lectura = valor de arranque del
  // DS18B20 (todavia no ha medido), no es una temperatura real.
  bool lecturaInvalida = (t == DEVICE_DISCONNECTED_C) || (!hayLectura && t ==
85.0);

  if (lecturaInvalida) {
```

```
    erroresSensor++;
    Serial.println("Error leyendo el sensor DS18B20");
    return;
  }

  erroresSensor = 0;
  hayLectura = true;
  tempActual = t;

  // % = posicion de la temperatura entre MIN (0%) y MAX (100%)
  porcentaje = (tempActual - tempMin) * 100.0 / (tempMax - tempMin);
  porcentaje = constrain(porcentaje, 0.0, 100.0);

  Serial.print("Temperatura: "); Serial.print(tempActual, 2);
  Serial.print(" C | ");          Serial.print(porcentaje, 0);
  Serial.print(" % | Estado: "); Serial.println(nombreEstado());
}

// ================================================================
// CICLO: CALENTANDO hasta MAX -> ENFRIANDO hasta MIN -> CALENTANDO ...
void actualizarCiclo() {
  // Seguridad: si el sensor falla varias veces seguidas, apagar foco y Peltier
  if (erroresSensor >= MAX_ERRORES_SENSOR) {
    if (estado != ERROR_SENSOR) cambiarEstado(ERROR_SENSOR);
    apagarFocoYPeltier();
    actualizarExtractor();
    return;
  }

  // Todavia no hay ninguna lectura valida: no hacer nada
  if (!hayLectura) return;

  switch (estado) {
    case INICIANDO:
    case ERROR_SENSOR:
      // Primera lectura valida (o el sensor se recupero): decidir por donde
empezar
      if (erroresSensor == 0) {
        cambiarEstado(tempActual >= tempMax ? ENFRIANDO : CALENTANDO);
      }
      break;

    case CALENTANDO:
      if (tempActual >= tempMax) cambiarEstado(ENFRIANDO);
      break;

    case ENFRIANDO:
      if (tempActual <= tempMin) cambiarEstado(CALENTANDO);
      break;
  }

  aplicarSalidas();
}

void cambiarEstado(EstadoCiclo nuevo) {
  estado = nuevo;
  momentoCambioEstado = millis();
  apagarFocoYPeltier();   // los dos apagados durante la pausa de cambio
  actualizarExtractor();  // el extractor cambia en el mismo instante
  Serial.print(">>> Cambio de estado: "); Serial.println(nombreEstado());
}

// Extractor: encendido solo al ENFRIAR (y en error del sensor, para sacar
```

```
// el calor que quede). Apagado al CALENTAR para no perder el calor del foco.
void actualizarExtractor() {
  bool encender = (estado == ENFRIANDO || estado == ERROR_SENSOR);
  digitalWrite(RELAY_EXTRACTOR, encender ? RELAY_ACTIVO : RELAY_INACTIVO);
}

void aplicarSalidas() {
  actualizarExtractor();

  // Durante la pausa despues de un cambio, foco y Peltier quedan apagados
  if (millis() - momentoCambioEstado < PAUSA_CAMBIO_MS) {
    apagarFocoYPeltier();
    return;
  }

  if (estado == CALENTANDO) {
    digitalWrite(RELAY_PELTIER, RELAY_INACTIVO);  // primero apagar el otro
    digitalWrite(RELAY_FOCO,    RELAY_ACTIVO);
  } else if (estado == ENFRIANDO) {
    digitalWrite(RELAY_FOCO,    RELAY_INACTIVO);
    digitalWrite(RELAY_PELTIER, RELAY_ACTIVO);
  } else {
    apagarFocoYPeltier();
  }
}

void apagarFocoYPeltier() {
  digitalWrite(RELAY_FOCO,    RELAY_INACTIVO);
  digitalWrite(RELAY_PELTIER, RELAY_INACTIVO);
}

const char* nombreEstado() {
  switch (estado) {
    case CALENTANDO:    return "CALENTANDO";
    case ENFRIANDO:     return "ENFRIANDO";
    case ERROR_SENSOR: return "ERROR SENSOR";
    default:            return "INICIANDO";
  }
}

// ============================================================================
// BOTONES
// Devuelve true solo en el instante en que el boton se presiona
bool botonPresionado(int pin, bool &anterior, unsigned long &ultimoPulso) {
  bool lectura = digitalRead(pin);
  bool presionado = false;
  if (lectura == LOW && anterior == HIGH && (millis() - ultimoPulso) >
DEBOUNCE_MS) {
    ultimoPulso = millis();
    presionado = true;
  }
  anterior = lectura;
  return presionado;
}

void manejarBotones() {
  // MODO: alterna entre editar MAX y editar MIN
  if (botonPresionado(BTN_MODO, anteriorModo, ultimoPulsoModo)) {
    modoEdicion = (modoEdicion == EDITAR_MAX) ? EDITAR_MIN : EDITAR_MAX;
    Serial.print("Editando: "); Serial.println(modoEdicion == EDITAR_MAX ? "Max"
: "Min");
  }
```

```
  // SUBIR
  if (botonPresionado(BTN_SUBIR, anteriorSubir, ultimoPulsoSubir)) {
    if (modoEdicion == EDITAR_MAX) {
      tempMax += PASO_SETPOINT;
      if (tempMax > LIMITE_SUPERIOR) tempMax = LIMITE_SUPERIOR;
    } else {
      tempMin += PASO_SETPOINT;
      if (tempMin > tempMax - MARGEN_MINIMO) tempMin = tempMax - MARGEN_MINIMO;
    }
    guardarConfiguracion();
  }

  // BAJAR
  if (botonPresionado(BTN_BAJAR, anteriorBajar, ultimoPulsoBajar)) {
    if (modoEdicion == EDITAR_MAX) {
      tempMax -= PASO_SETPOINT;
      if (tempMax < tempMin + MARGEN_MINIMO) tempMax = tempMin + MARGEN_MINIMO;
    } else {
      tempMin -= PASO_SETPOINT;
      if (tempMin < LIMITE_INFERIOR) tempMin = LIMITE_INFERIOR;
    }
    guardarConfiguracion();
  }
}

// Asegura valores validos (por si en memoria habia algo raro)
void validarLimites() {
  tempMax = constrain(tempMax, LIMITE_INFERIOR + MARGEN_MINIMO, LIMITE_SUPERIOR);
  tempMin = constrain(tempMin, LIMITE_INFERIOR, tempMax - MARGEN_MINIMO);
}

void guardarConfiguracion() {
  prefs.begin("configTemp", false);
  prefs.putInt("tempMax", tempMax);
  prefs.putInt("tempMin", tempMin);
  prefs.end();
  Serial.print("MAX: "); Serial.print(tempMax); Serial.print(" C  MIN: ");
Serial.print(tempMin); Serial.println(" C");
}

// ================================================================
// LCD 16x2
//   Fila 0: ">Max:60° Min:30°"  (la ">" marca el valor que se edita)
//   Fila 1: "45.3°  50% Calor"  (o "Error sensor" si el sensor falla)
void actualizarLCD() {
  char linea[17];

  snprintf(linea, sizeof(linea), "%cMax:%2d%c%cMin:%2d%c",
           modoEdicion == EDITAR_MAX ? '>' : ' ', tempMax, GRADO,
           modoEdicion == EDITAR_MIN ? '>' : ' ', tempMin, GRADO);
  imprimirLinea(0, linea);

  if (estado == ERROR_SENSOR) {
    imprimirLinea(1, "Error sensor");
  } else if (!hayLectura) {
    imprimirLinea(1, "Leyendo sensor..");
  } else {
    const char* textoEstado;
    if (estado == CALENTANDO)     textoEstado = "Calor";
    else if (estado == ENFRIANDO) textoEstado = "Frio";
    else                          textoEstado = "";

    snprintf(linea, sizeof(linea), "%4.1f%c %3d%% %s",
```

```
              tempActual, GRADO, (int)(porcentaje + 0.5), textoEstado);
    imprimirLinea(1, linea);
  }
}

// Escribe un texto en la fila indicada y rellena con espacios hasta 16
// caracteres, para borrar restos de textos anteriores sin parpadeo
void imprimirLinea(int fila, const char* texto) {
  lcd.setCursor(0, fila);
  int n = 0;
  while (texto[n] != '\0' && n < 16) {
    lcd.print(texto[n]);
    n++;
  }
  while (n < 16) {
    lcd.print(' ');
    n++;
  }
}
```

Resultados

La instalación de todos los componentes se realizó correctamente y las conexiones en la protoboard fueron verificadas antes de energizar el circuito. Posteriormente, el código fue cargado con éxito en la ESP32 sin presentar errores de compilación ni de comunicación.

Durante las pruebas iniciales, el sistema funcionó de manera adecuada. Más adelante, se modificó el programa para establecer un rango de temperatura de trabajo entre 30 °C y 40 °C. Bajo esta configuración, cuando la temperatura dentro de la caja alcanza los 40 °C**,** el foco utilizado como fuente de calor se apaga automáticamente y se activan los ventiladores junto con la celda Peltier para iniciar el proceso de enfriamiento.

El sistema continúa reduciendo la temperatura interna hasta alcanzar los 30 °C, valor definido como límite mínimo. Una vez alcanzada esta temperatura, el control mantiene las condiciones establecidas para conservar la caja dentro del rango programado. De esta manera, se logra un control automático de temperatura mediante el uso de la ESP32, garantizando ciclos de calentamiento y enfriamiento según los parámetros definidos.

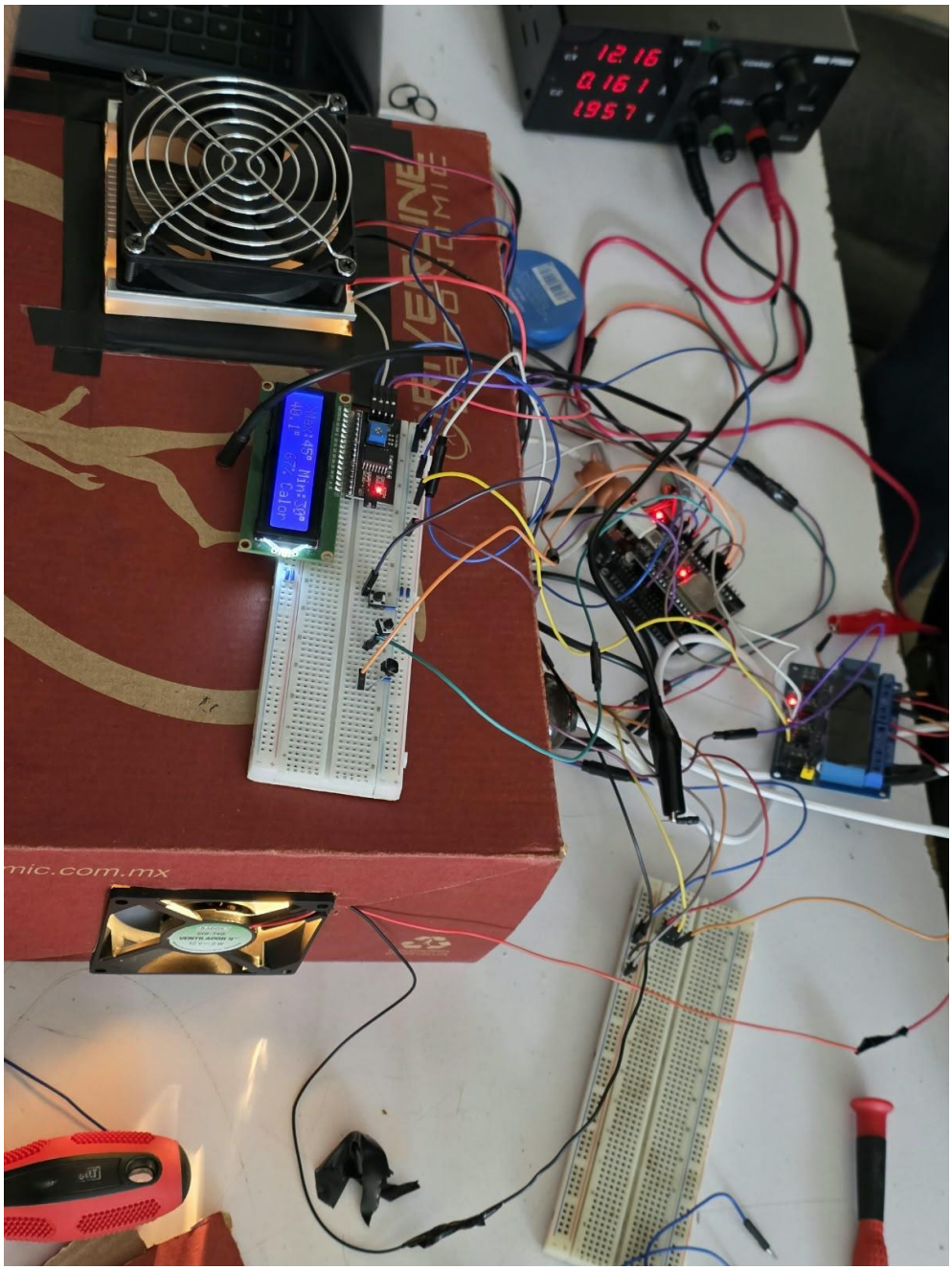


Ilustración 2Circuito todo ensamblado y en ejecución

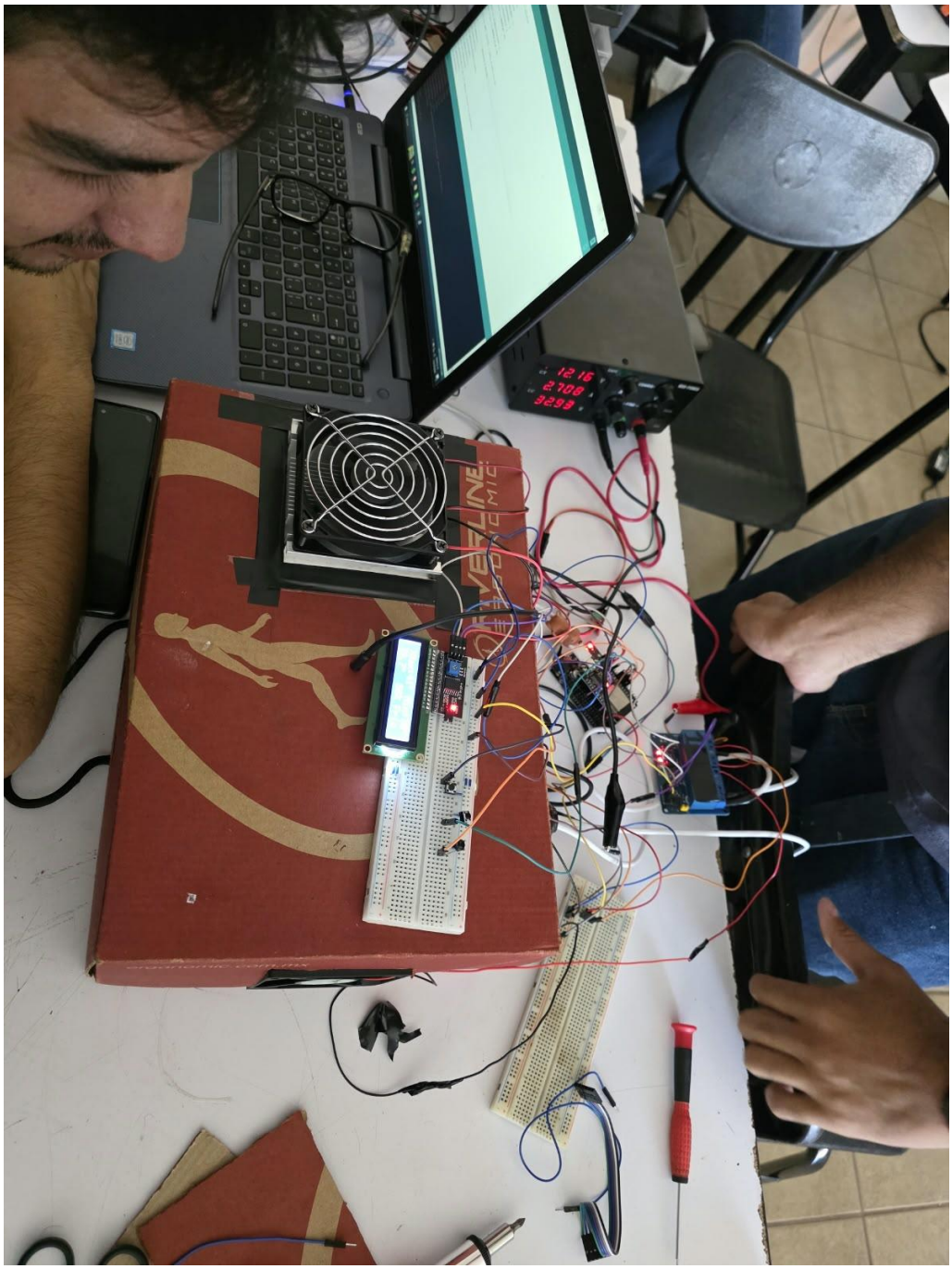


Ilustración 3Circuito en ejecución enfriando y calentado visto todo ensamblado

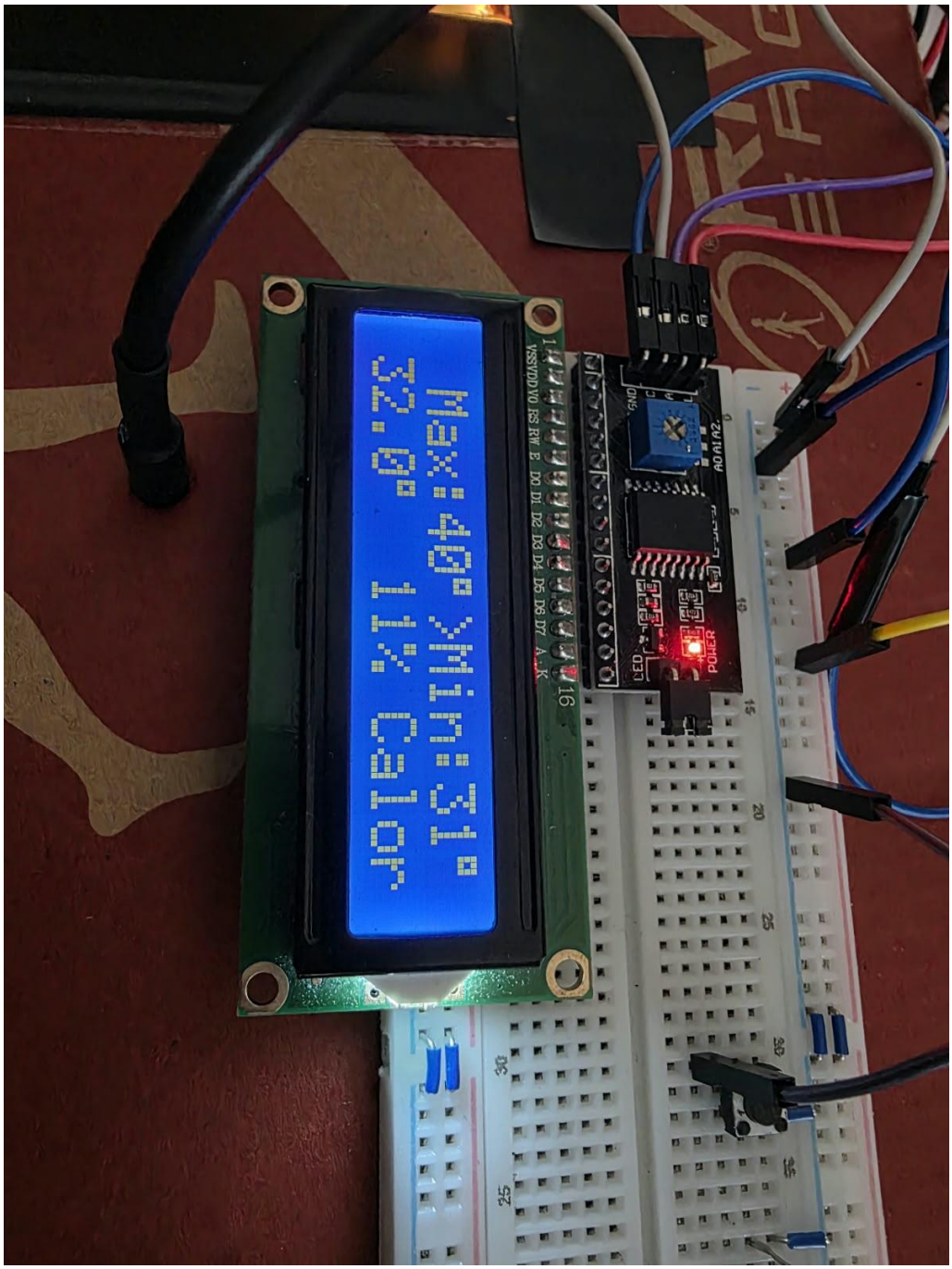


Ilustración 4Pantalla lcd con los máximos y mínimos de enfriamiento y calentamiento

Observaciones y Conclusiones

Se logró implementar correctamente un sistema de control automático de temperatura utilizando una ESP32, una celda Peltier, ventiladores y un foco como fuente de calor. Durante las pruebas, todas las conexiones y componentes funcionaron adecuadamente, permitiendo que el sistema respondiera de manera eficiente a los cambios de temperatura. La programación estableció un rango de operación entre 30 °C y 40 °C, logrando que al alcanzar la temperatura máxima se desactivara el foco y se activara el sistema de enfriamiento. Asimismo, cuando la temperatura descendía a 30 °C, el sistema mantenía las condiciones programadas. Con ello, se comprobó que la automatización mediante la ESP32 permite un control preciso y confiable de la temperatura, cumpliendo satisfactoriamente con los objetivos planteados para la práctica.

Referencias

[1] Espressif Systems. (s. f.). ESP32 Series Datasheet. Espressif Systems.

[2] Arduino. (s. f.). Arduino Documentation. Arduino.

[3] Gomar, J. (2018, 16 octubre). *Que es la célula Peltier y cómo funciona*. Profesional

[4] *Tutorial sensor digital de temperatura DS18B20*. (s. f.). Naylamp Mechatronics - Perú.